

AI Media Generation API

1. Project Overview

AI Media Generation is a FastAPI-based service responsible for generating audio and visual media from scene-based scripts.

The service converts scene narration into speech, selects an appropriate voice, generates AI-based visuals from visual prompts, and creates scene-level video assets.

The project focuses on clean provider abstraction, API integration, media generation, local media storage, input validation, error handling, automated testing, and integration with the overall video-generation workflow.

2. Objective

The objective of this module is to implement AI-generated audio and visual media required for video generation.

The module supports:

- Text-to-Speech
- Voice selection
- Scene-wise narration
- Audio generation
- Audio preview
- AI image generation
- AI video generation
- Scene-based visual generation
- Local media storage
- REST APIs
- Provider abstraction
- Error handling
- Automated testing

The project does not build an AI video-generation model from scratch. Instead, it integrates available providers and models through a clean application architecture.

3. Responsibilities

Voice

- Text-to-Speech
- Voice selection
- Scene-wise narration
- Audio generation
- Audio preview

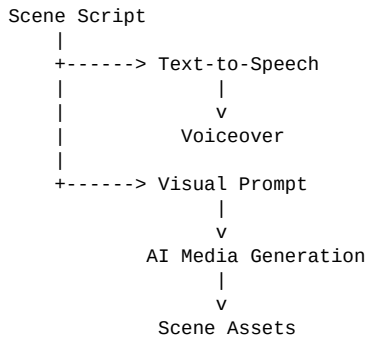
AI Visuals

Where practical, the module integrates available APIs/models for AI image generation, AI video generation, and scene-based visual generation.

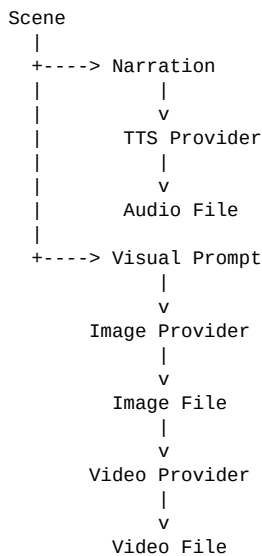
Optional Features

Avatar generation, talking avatar, and lip-sync are optional and are not required for the core implementation.

4. Expected Workflow



Detailed Media Generation Workflow



5. Features

- Text-to-Speech generation
- Tamil and English voice support
- Male and female voice selection
- Scene-wise narration
- AI image generation using Hugging Face
- Local video generation using FFmpeg
- Scene-based media generation
- Audio preview through media URLs
- Local media storage
- REST API integration
- Provider abstraction
- Input validation
- Error handling
- Automated testing

6. Technologies Used

Python, FastAPI, Pydantic Settings, Edge TTS, Hugging Face, FFmpeg, ImageIO FFmpeg, and Pytest.

7. Project Structure

```

ai-media-generation/
|
+-- app/
|   +-- api/routes/
|   +-- core/config.py
|   +-- providers/

```

```

| | +-- image/
| | +-- tts/
| | +-- video/
| +-- schemas/
| +-- services/
| +-- storage/
|
+-- media/
| +-- audio/
| +-- images/
| +-- videos/
|
+-- tests/
| +-- test_image.py
| +-- test_storage.py
| +-- test_tts.py
| +-- test_video.py
|
+-- .env
+-- README.md
+-- requirements.txt

```

8. Provider Architecture

```

ImageProvider
|
+-- ExistingImageProvider
+-- HuggingFaceImageProvider

TTSPProvider
|
+-- EdgeTTSPProvider

VideoProvider
|
+-- LocalVideoProvider

```

The application separates media-generation logic from individual provider implementations. The provider factory selects the configured provider, allowing providers to be changed without changing the main media-generation workflow.

9. Configuration

```

APP_NAME=AI Media Generation API
APP_VERSION=1.0.0
ENVIRONMENT=development
BASE_URL=http://127.0.0.1:8000
IMAGE_PROVIDER=huggingface
VIDEO_PROVIDER=local
TTS_PROVIDER=edge
HF_TOKEN=your_huggingface_token
ELEVENLABS_API_KEY=
ELEVENLABS_TTS_MODEL=eleven_multilingual_v2

```

API keys and tokens must not be committed to source control. Use a local `.env` file for development and a `.env.example` file with placeholders for shared repositories.

10. Installation

```

python -m venv venv

# Windows PowerShell
.\venv\Scripts\Activate.ps1

pip install -r requirements.txt

# Start the API
uvicorn app.main:app --reload

```

API: <http://127.0.0.1:8000>

Swagger documentation: <http://127.0.0.1:8000/docs>

11. API

The current media-generation input structure is:

```
{
  "project_id": "project-001",
  "scenes": [
    {
      "scene_id": 1,
      "narration": "A farmer begins his work early in the morning.",
      "visual_prompt": "A realistic farmer working in a green agricultural field during sunrise.",
      "voice": "male"
    }
  ]
}
```

Field	Description
project_id	Identifier for the project
scene_id	Identifier for an individual scene
narration	Text used for scene narration
visual_prompt	Prompt used for visual generation
voice	Voice preference for narration

Example response:

```
{
  "project_id": "project-001",
  "scenes": [
    {
      "scene_id": 1,
      "audio_path": "media/audio/scene_1.mp3",
      "image_path": "media/images/scene_1.png",
      "video_path": "media/videos/scene_1.mp4"
    }
  ]
}
```

The exact team-wide API response contract will be finalized during integration with the upstream scene-planning module.

12. Media Storage

Generated media is stored locally.

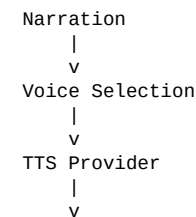
Structure:

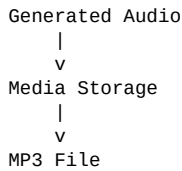
media/audio/
media/images/
media/videos/

The storage layer provides separate methods for saving audio, images, and videos and validates filenames to prevent invalid file paths.

13. Audio Generation

The TTS service receives narration text, a voice ID, and an output filename. It validates the input, calls the configured TTS provider, and stores the generated audio.

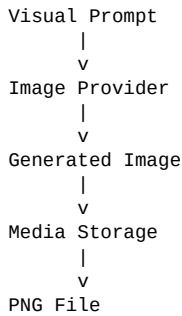




14. Voice Selection

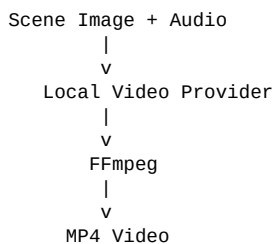
The system supports voice selection for scene narration. The selected voice is passed to the configured TTS provider, allowing different voices to be used for different scenes.

15. AI Image Generation



The project currently supports Hugging Face as an AI image provider and also supports an existing-image provider.

16. Video Generation



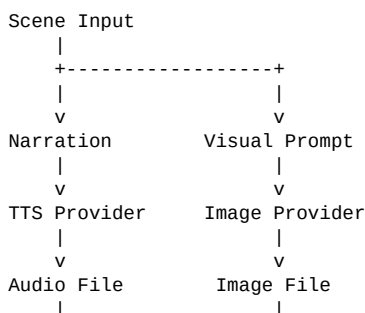
The local video provider can combine a scene image with narration audio. Audio is explicitly mapped into the generated MP4 when audio is provided.

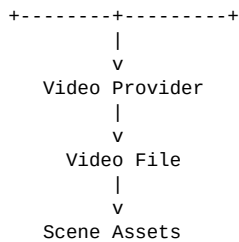
17. Media Preview

http://127.0.0.1:8000/media/audio/scene_1.mp3
http://127.0.0.1:8000/media/images/scene_1.png
http://127.0.0.1:8000/media/videos/scene_1.mp4

These URLs can be opened in a browser while the FastAPI server is running and can be consumed by other modules or the frontend during team integration.

18. Scene Media Generation





19. Error Handling

The application validates input and handles failures from media providers. Examples include empty narration, empty voice ID, empty image prompt, empty filename, missing image/audio files, invalid filenames, empty provider responses, provider failures, and video generation failures.

Application-specific exceptions include: TTSGenerationError, ImageGenerationError, MediaGenerationError, ProjectGenerationError, and ProviderConfigurationError.

20. Automated Testing

```

tests/
|
+-- test_image.py
+-- test_storage.py
+-- test_tts.py
+-- test_video.py

```

```

Run:
python -m pytest -v

```

```

Test Result:
20 passed in 27.53s

```

```

20 tests
20 passed
0 failed

```

The image-generation unit tests use fake providers, so the automated service tests do not consume Hugging Face generation credits.

21. Team Integration

The AI Media Generation module is designed to receive scene information from the upstream scene-planning workflow.

```

{
  "project_id": "project-001",
  "scenes": [
    {
      "scene_id": 1,
      "narration": "Scene narration",
      "visual_prompt": "Visual description",
      "voice": "male"
    }
  ]
}

```

```

Scene Input
|
| v
AI Media Generation
|
+----> Audio
+----> Image
+----> Video
|
| v
Scene Assets

```

These generated assets can be consumed by downstream modules such as the video editor and frontend. The exact final team-wide input/output contract will be aligned with the upstream scene-planning module.

22. Optional Features

Avatar Generation: Generates a digital character/avatar.

Talking Avatar: Animates an avatar so that it appears to speak the generated narration.

Lip-sync: Synchronizes the avatar's mouth movements with generated speech.

These features are optional and are not required for the core implementation.

23. Limitations

- Local media storage is currently used.
- Media URLs are accessible through the running FastAPI server.
- Local video generation depends on FFmpeg.
- AI image generation depends on the configured external provider.
- External AI providers may have usage limits or availability constraints.
- The final team-wide input/output contract is pending integration with the upstream scene-planning module.

24. Future Improvements

- Cloud-based media storage
- Additional TTS providers
- Additional image-generation providers
- Advanced AI video providers
- Avatar generation
- Talking avatar support
- Lip-sync
- Background music generation
- Production authentication
- Media cleanup and lifecycle management
- Database-based media metadata management

25. Conclusion

The AI Media Generation API provides a modular service for generating scene-level audio and visual media.

The implementation provides TTS integration, voice selection, AI image generation, local video generation, scene-based media generation, media storage, REST APIs, provider abstraction, input validation, error handling, automated testing, and media preview.

The automated test suite currently contains 20 tests, and all 20 tests pass successfully.

The module is designed to integrate with the other project modules through a defined scene-based API contract.